

General Introduction to Hardware Verification

SyoSil, 2026

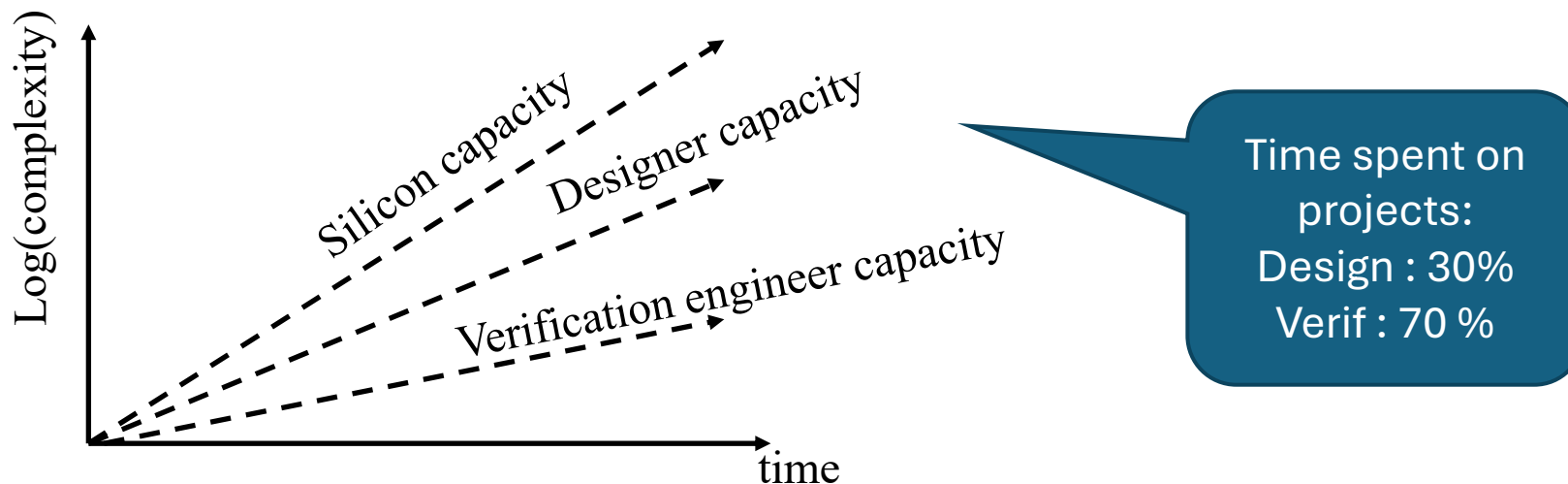
Contents

- Introduction
 - Who are We
 - Practical information
- Why Verification
- What is Verification
- What are We Verifying
- What is Functional Verification

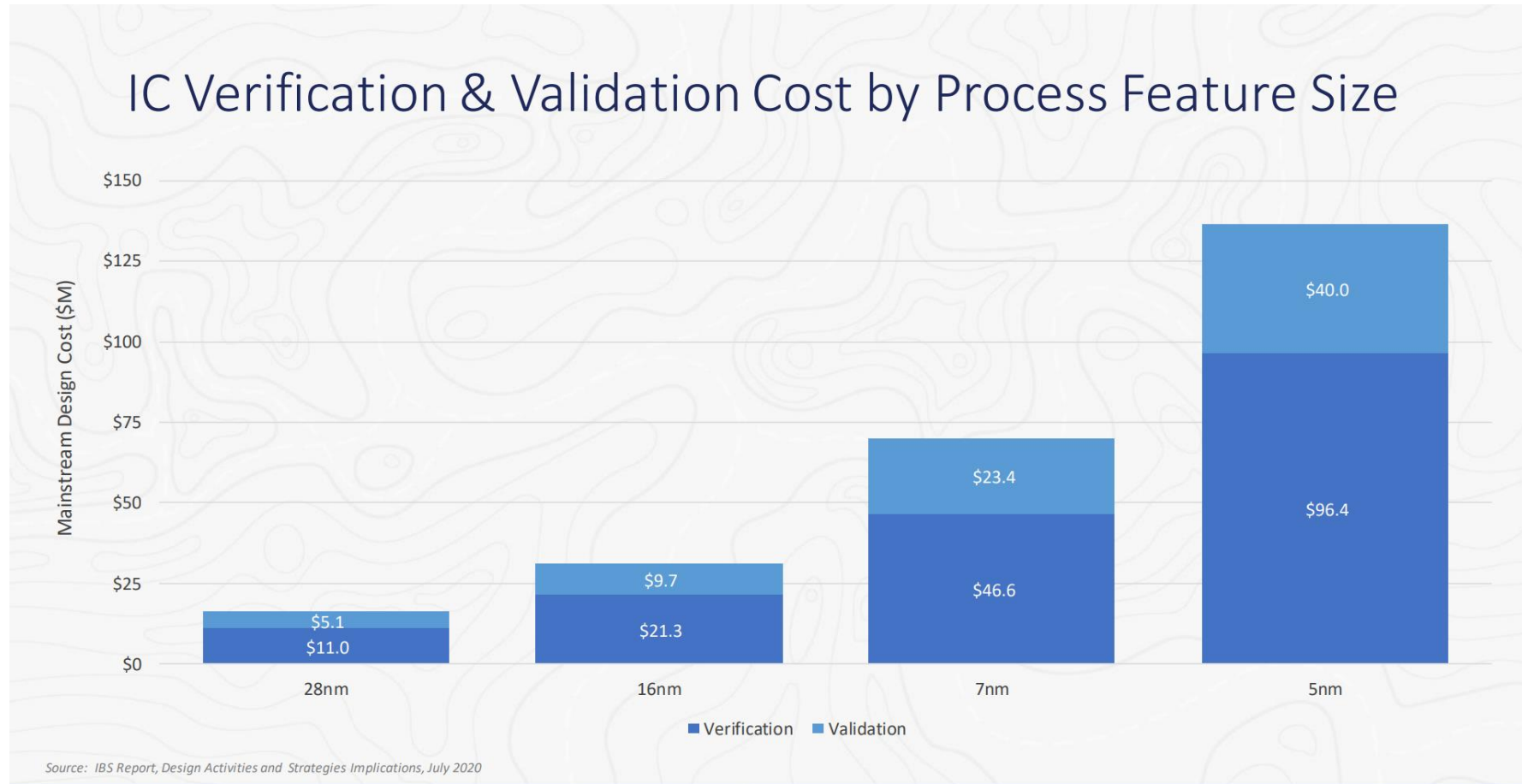
Why Verification? (of hardware)

Digital Verification Becomes Bottleneck

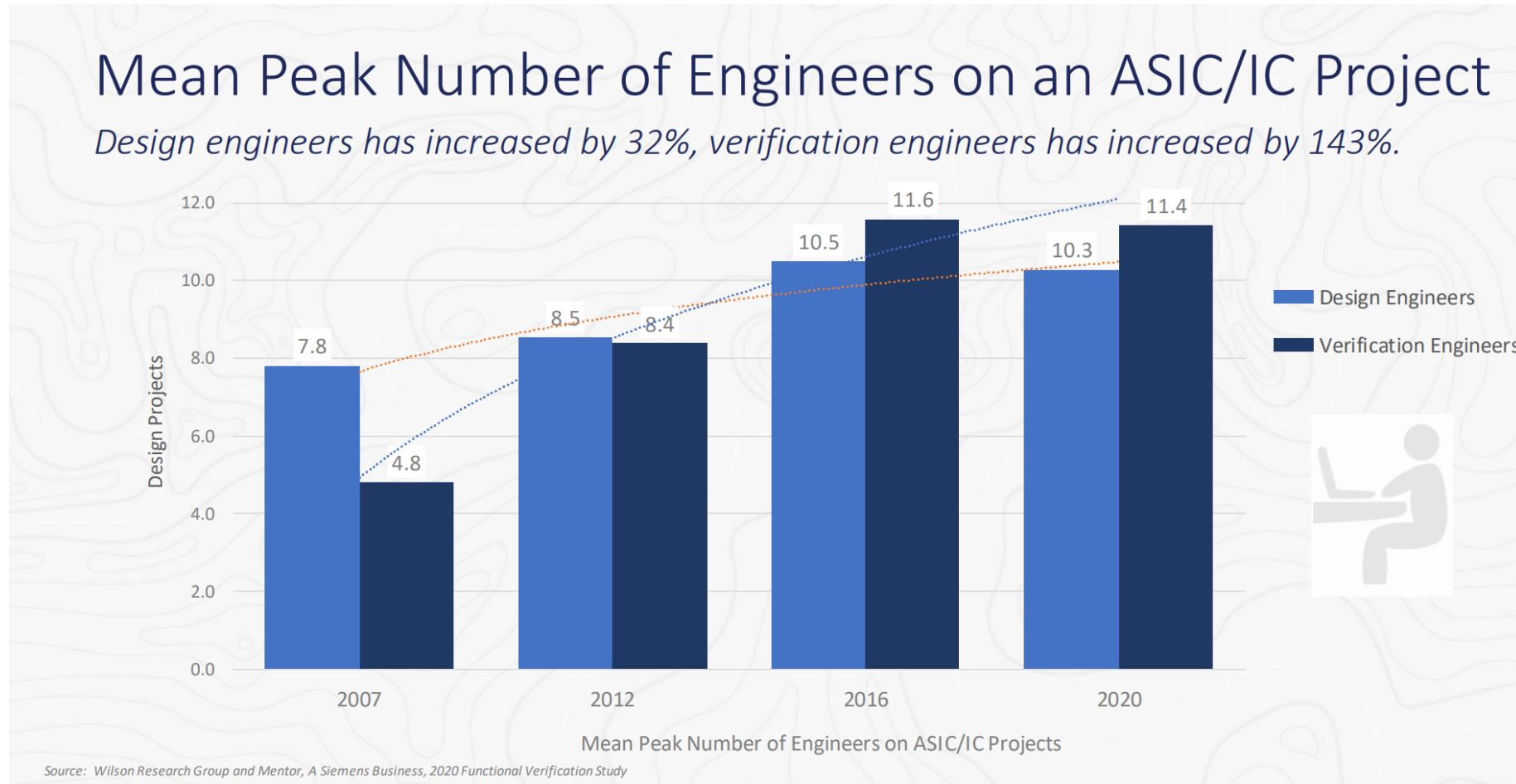
- Silicon capacity is constantly increasing
- Design gap: Designers cannot keep up
- Verification gap: Verification engineers can't even keep up with designers.
- Majority of chip respins is due to functional bugs !



Wilson research group result 2020



Wilson research group result 2020



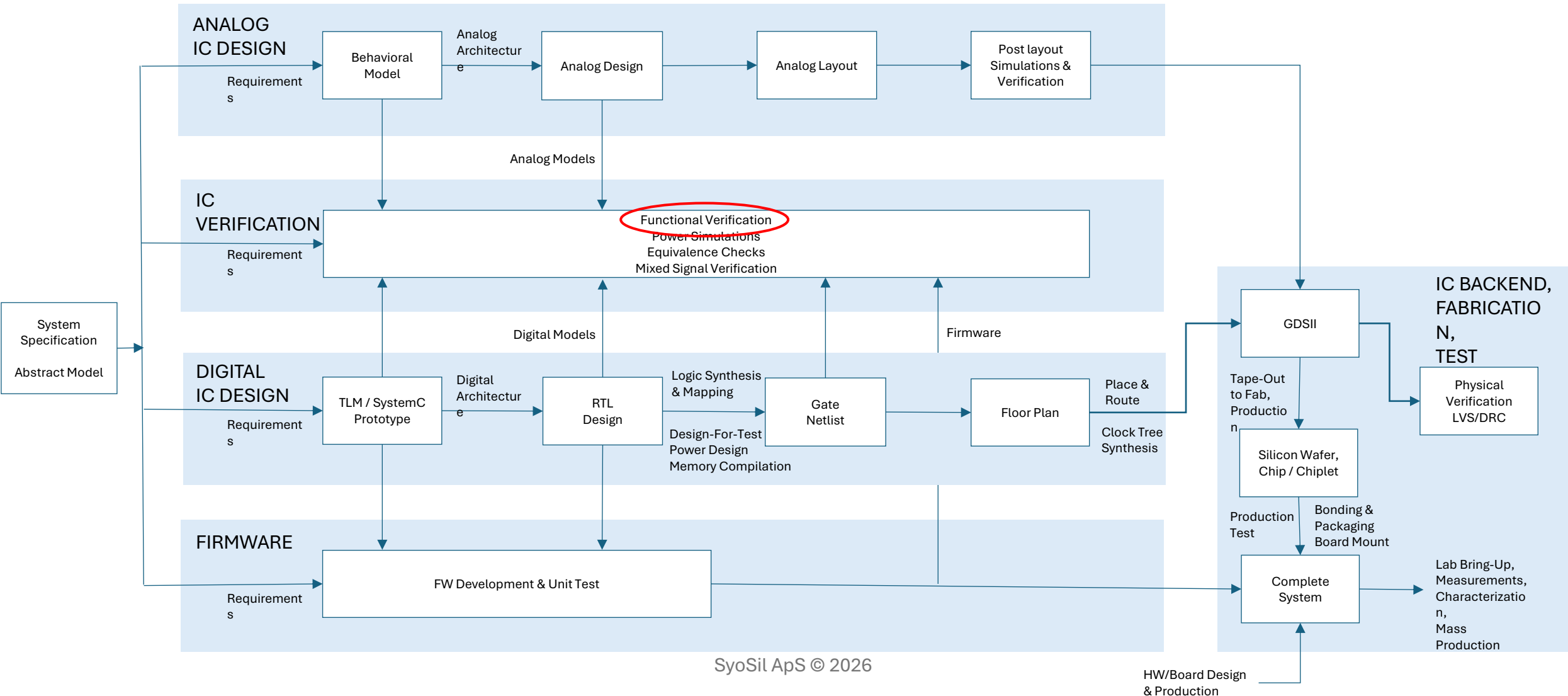
Why Verification?

- High quality verification can save the company a lot of money and resources
- Proper verification reduce the time-to-market
- For the engineer
 - Difficult and complex 😊

What is Verification? (of hardware)

What is Verification?

- To be precise
 - Digital Chip Verification (IC, ASIC, ...)
 - Functional verification of the RTL design



What does it require?

- Many technologies
 - OOP & Use of constraint solver technologies
 - Formal technology
 - Big data (coverage, regressions)
 - AI (More tools for verification starts to utilize AI/ML)
- How do we know it is complete?
- To overcome the complexity
 - OOP SW frameworks (Utilizing design patterns)
 - Model Driven SW Development (Code Generation)
 - Reusable verification environments at different system levels and abstraction levels
 - Formal techniques

Real life numbers - Example

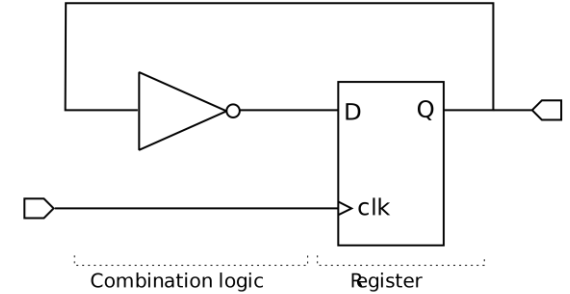
- People:
 - 30 ASIC engineers
 - 60 Algorithm/Electronics engineers
 - 300 SW engineers (FW, Drivers etc.)
- 80.000.000 SEK
- 45 git repos
- 167 Testbenches
- SystemC TLM, VMM, UVM, SystemVerilog, C/C++, Python

Major Verification Methodologies

- **#1: Constrained Random / Coverage Driven Verification (CRV / CDV)**
 - Simulation based, main stream verification techniques.
 - Object oriented test benches for better reuse
- **#2: Standardized Verification IP (VIP)**
 - Agent, reference model, scoreboard, coverage model etc.
 - Modularity and reuse.
- **#3: Formal property checking**
 - Does not require a test bench to find bugs.
 - Assertion Based Verification (ABV). Automatically use algorithms to verify functionality
 - Complement simulation based techniques

What are We Verifying?

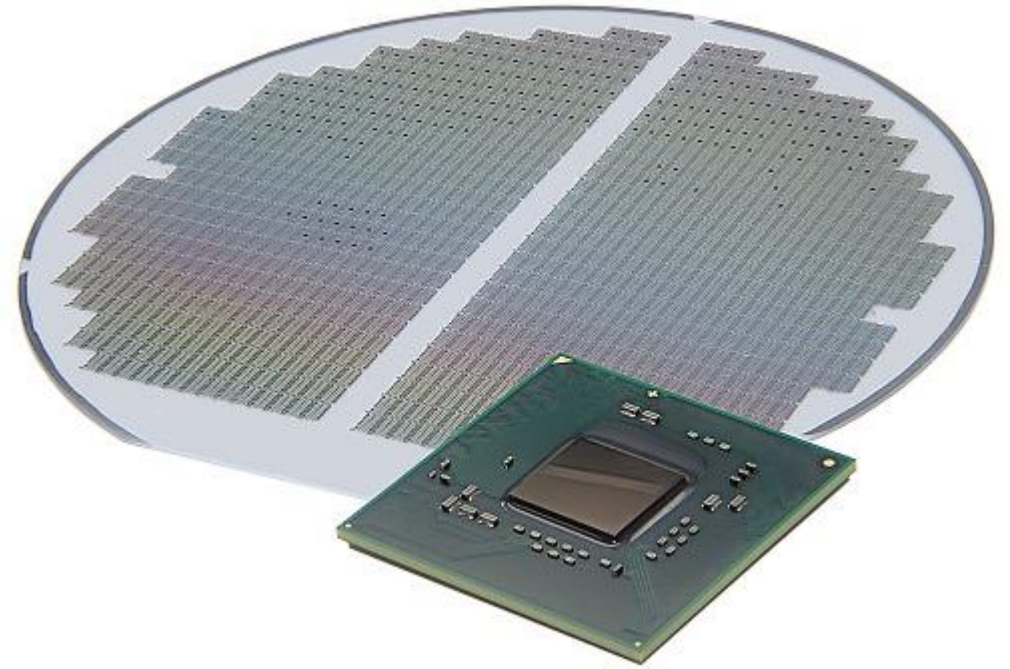
What is an RTL Module



- From Wikipedia:
 - In digital circuit design, register-transfer level (RTL) is a design abstraction which models a synchronous digital circuit in terms of the flow of digital signals (data) between hardware registers, and the logical operations performed on those signals.
- Described abstractly in a Hardware Description Language (HDL)
 - Chiesel
 - VHDL
 - SystemVerilog
- Can be written from scratch or bought finished known as an IP
 - IP=Intellectual Property
- Usually synthesized into a gate representation to define actual hardware

Example 😊

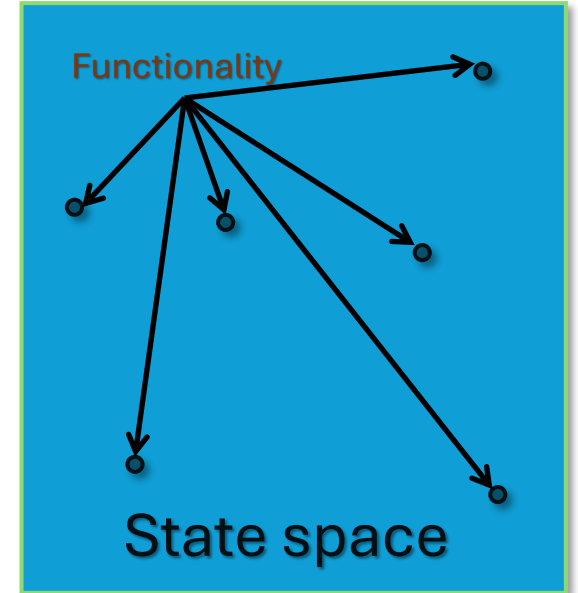
- **Name:** Cerebras WSE-3
- **Transistors:** 4 trillion
- **AI-Optimized Cores:** 900,000
- **On-Chip Memory:** 44 GB SRAM
- **Peak AI Compute:** 125 petaflops
- **Memory Bandwidth:** 21 petabytes per second
- **Manufacturing Process:** TSMC 5nm
- **Chip Size:** 46,250 square millimeters (the largest AI chip ever built)



What is Functional Verification?

What is functional verification?

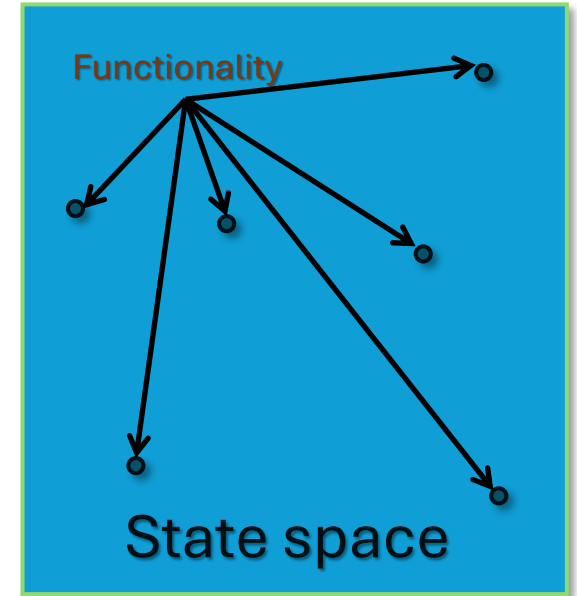
- **Device**
 - Typically, a digital design expressed as RTL using an HDL
 - Defines a functional state space
- **Functional verification**
 - **Stimuli**
 - Provide input to device
 - **Correctness**
 - Will the device work as intended within the functional state space?
 - I/O
 - Data transformations
 - Configurations, modes of operation
 - Performance
 - ...
 - **Completeness**
 - How well is the functional state space traversed



Verification Quality

- **Completeness wrt. functional state space**
- **Functional correctness**

Functional State Space Example



- Device
 - Adder with 2, 8bit, ports, a and b
 - One 9 bit output port
 - No internal state
 - Number of states in functional state space:
 - Input (for each clk):
 - rst: 2^1
 - a: 2^8
 - b: 2^8
 - Total: $2 \cdot 256 \cdot 256 = 131072$
 - Output: $2^9 = 512$

```
module adder(  
    input    logic clk,  
    input    logic rst,  
    input    logic [7:0] a,  
    input    logic [7:0] b,  
    output   logic [8:0] c);  
    always @(posedge clk) begin  
        if(rst == 1'b0) begin  
            c <= 0;  
        end else begin  
            c <= a+b;  
        end  
    end  
end  
endmodule
```

Traditional verification methodology

- Typically uses a simulation based approach
 - **Stimuli:** Directed tests
- Manually implementing
 - Power on, reset
 - Configurations
 - Many test vectors
- **Correctness** (Functional)
 - Manual inspection of log and waves
 - Pass/fail built-in to tests
 - Sometimes post processing of log files
- **Completeness**
 - Verification plan → List of directed tests to be implemented
 - When all tests are done then we are done

Verification Quality
Stimuli: - Manual
Correctness: - Manual or semi automated - Do not scale well
Completeness: - Implicit completeness criteria

Problems with directed tests

- **Stimuli**
 - Tests are implemented manually
- **Correctness**
 - Manual inspection is tedious and prone to human error
- **Completeness**
 - Functional state space is too big, and verification engineers are expensive
 - Need to implement many tests
 - Grows exponentially as device functionality grow (Functional state Space Explosion)
 - Only the scenarios imagined by the verification engineer are verified
 - Poor scalability as verification quality over time decreases as the completeness criteria gets harder to fulfill
- Test Implementation is One Dimensional (All in one test)
 - **Correctness**
 - Functional checks as they are typically built-in into the tests
 - **Completeness**
 - Functional state space coverage implicit to test
 - Device changed → test might not cover the correct functional state space anymore
 - Poor re-usability of testcases and test harness

Verification methodology switch - Why?

- What do we really want to achieve?
 - Have the best possible **verification quality** which scales with device complexity
 - How to achieve best possible **verification quality**?
 - Verify functional **correctness** for the complete functional state space if possible (**completeness**).
 - Metric for functional state space **completeness**?
 - The functional state space can be expressed as functional coverage
 - Thus, the task can be rephrased to: **Can we fulfill the functional coverage easier?**
 - Novel idea: Let the computer do the work (write the tests - **Stimuli**)!
- **NOTE:** Now, the **verification quality** is tied to the **correctness** of the functional coverage model (and checkers) and not directly to the amount of implemented tests (**Stimuli**)

Novel idea: Constrained Random Verification (CRV)

- AKA Coverage Driven Verification - CDV
- When the machines have to write the tests then **correctness**, **completeness** and **stimuli** in a test approach is insufficient
- The 1D approach has to be split into individual dimensions (3D) as the machine will write test by generating stimuli not knowing the **completeness** and **correctness** criterias.
- Thus, to produce a machine written test then the machine needs to be able to independently:
 - Produce **stimuli**
 - Check/report **correctness**
 - Check/report **completeness**

Only the Scenarios Imagined by Verif Engineers ...

Fix it now !

Knowns Unknowns	Known Knowns <i>Things we are aware of and understand.</i>	Known Unknowns <i>Things we are aware of but don't understand.</i>
	Unknown Knowns <i>Things we understand but are not aware of.</i>	Unknown Unknowns <i>Things we are neither aware of nor understand.</i>
	Knowns	Unknowns

Debug debug debug...
but do you have a
testcase?

What most people
can imagine and
look for...

Will NEVER be
found using
directed testing